SIMATS ENGINEERING

CO4-ASSESSMENT TOOL1- INDUSTRY PROBLEM TASK

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1709**

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning,*
and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 20%

Time Duration : 1 Hour

QUESTIONS: 4 **Total: Marks:40**

Code of Conduct:

I Dillilokesh D 192311097 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Dillilokesh D

INDUSTRY SCENARIO

Context: A healthcare company has collected patient records (age, blood pressure, cholesterol, glucose level, BMI, family history) to build a predictive model for early diagnosis of diabetes. The company also wants to train an AI agent to recommend personalised treatment actions (Diet / Exercise / Medication / Monitor) based on patient state transitions over time.

You are hired as an AI/ML Engineer. Address the following tasks:

Task 1: Data Preparation & Inductive Learning

(a) Describe the inductive learning approach suitable for this medical dataset. Identify the target variable and at least three key features with justification.

(b) Explain how you would handle class imbalance (more non-diabetic than diabetic cases) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).

(c) Split the dataset (70:30) and justify the split ratio for a medical use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

Task 2: Decision Tree for Diagnosis

(a) Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.

(b) Evaluate the model: report Accuracy, Precision, Recall, and F1-Score. Identify False Negatives (missed diabetes cases) and suggest how to reduce them—justify clinically.

(c) Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment in a clinical setting.

Task 3: Statistical Learning for Risk Stratification

(a) Apply Naïve Bayes or Logistic Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.

(b) Perform feature importance analysis. Identify the top 3 predictors of diabetes from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

Task 4: Reinforcement Learning for Treatment Recommendation

(a) Model the treatment recommendation as an MDP. Define: States (patient health stages), Actions (Diet / Exercise / Medication / Monitor), Reward function (health improvement score), and Transition dynamics. Justify each component.

(b) Apply Q-Learning to train the agent for at least 5 patient episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.

(c) Extract the final learned policy. Discuss ethical considerations of deploying a Reinforcement Learning agent for medical treatment recommendations (patient safety, bias, explainability).

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

ANSWERS

Introduction

A healthcare company has collected patient records containing age, blood pressure, cholesterol, glucose level, BMI and family history, with the goal of building a predictive model for early diagnosis of diabetes. In addition, the company wants an AI agent that recommends personalised treatment actions - Diet, Exercise, Medication or Monitor - based on how a patient's health state changes over time. As the AI/ML Engineer on this project, this report covers four tasks: (1) data preparation and the inductive learning approach, (2) building and evaluating a Decision Tree classifier, (3) applying a statistical learning model for risk stratification and comparing it with the tree, and (4) modelling treatment recommendation as a Markov Decision Process solved using Q-Learning. Since an actual raw dataset file was not supplied with this assignment, illustrative values consistent with typical clinical diabetes datasets (similar in spirit to the Pima Indians Diabetes dataset) are used throughout to demonstrate the methodology; the same steps apply directly once the real hospital data is plugged in.

Task 1: Data Preparation & Inductive Learning

1(a) Inductive Learning Approach

Inductive learning is the process by which a model generalises a decision rule or hypothis from a finite set of labelled training examples, so that it can predict the label of new, unseen examples. Since the hospital already has past patient records where the outcome (diabetic / non-diabetic) is known, this is a classic case of supervised inductive learning - the algorithm is given many (features -> label) pairs and it induces a general mapping function f(X) -> Y that is expected to generalise to future patients. This is suitable here because:

- The dataset is labelled (each patient record has a confirmed diagnosis), so supervised learning can be used directly instead of unsupervised clustering.
- The target concept (diabetes onset) is believed to follow consistent statistical patterns across patients (e.g. high glucose + high BMI + family history), which inductive algorithms like Decision Trees and Logistic Regression can capture as generalizable rules.
- Once trained, the induced hypothesis can be applied instantly to new patients at the time of screening, which is exactly the early-diagnosis use case the company wants.

Target Variable: Diabetes_Status - a binary class label (1 = Diabetic, 0 = Non-Diabetic). This is the variable the model is inductively learning to predict from the other patient attributes.

Key Features and Justification (at least three, five shown for completeness):

Feature	Type	Clinical / Statistical Justification
Glucose Level	Numeric	Direct physiological indicator of diabetes; fasting/plasma glucose is the primary diagnostic criterion used by clinicians, so it is expected to be the strongest predictor.
BMI (Body Mass Index)	Numeric	Obesity is a well-established risk factor for Type-2 diabetes due to insulin resistance; higher BMI correlates strongly with diabetic onset.
Age	Numeric	Risk of Type-2 diabetes increases with age as insulin sensitivity naturally declines over time.
Family History	Categorical (Yes/No)	Diabetes has a genetic/hereditary component; a positive family history significantly raises prior probability of the disease.
Blood Pressure	Numeric	Hypertension frequently co-occurs with diabetes (metabolic syndrome), making it a useful correlated feature.

1(b) Handling Class Imbalance

In most hospital screening datasets, non-diabetic cases far outnumber diabetic cases (e.g. a typical split may be around 70% non-diabetic vs 30% diabetic, and can be even more skewed in general-population screening data). If this imbalance is not addressed, the model tends to become biased towards the majority (non-diabetic) class - it can achieve high overall accuracy simply by predicting 'non-diabetic' most of the time, while missing a large fraction of actual diabetic patients (false negatives). In a medical context this is dangerous because a missed diagnosis delays treatment.

Illustrative imbalance before correction:

Class	Number of Records	Percentage
Non-Diabetic (0)	500	65%
Diabetic (1)	268	35%
Total	768	100%

Balancing strategies considered:

- Random Under-sampling: remove majority-class samples until classes are balanced. Simple, but discards potentially useful information and can hurt performance on a moderately-sized dataset such as this one.

- Random Over-sampling: duplicate minority-class samples. Easy to implement, but risks overfitting since the same diabetic records are repeated exactly.
- SMOTE (Synthetic Minority Over-sampling Technique): generates new synthetic minority-class samples by interpolating between existing diabetic patients and their nearest neighbours in feature space, instead of duplicating rows.
- Class-Weight Adjustment: keep the dataset as-is, but assign a higher misclassification penalty (class_weight='balanced') to the minority class during model training, so the loss function pushes the model to pay more attention to diabetic cases.

Chosen Strategy: SMOTE combined with class-weighting is recommended. SMOTE is preferred over plain under/over-sampling because it enriches the minority class with realistic synthetic examples rather than losing data or exactly duplicating records, which reduces overfitting risk. Class weighting is added as a lightweight complementary step for the classifier's cost function so that recall on diabetic patients is prioritised over raw accuracy - this directly matches the clinical priority of minimising missed diagnoses. After SMOTE, the training data reaches an approximately 50:50 balance, which is applied only on the training split (never on the test split, to keep evaluation realistic).

1(c) Train-Test Split and Preprocessing

The dataset is split 70:30 (70% training, 30% testing) using a stratified split so that the ratio of diabetic to non-diabetic cases is preserved in both subsets.

Justification of 70:30 for a medical use-case: Medical datasets are usually small to moderate in size (hundreds to a few thousand records) compared to typical industrial datasets, so a reasonably large test set (30%) is kept aside to obtain a statistically reliable estimate of clinical performance (accuracy, recall, AUC) before any real-world deployment, while still leaving 70% of the data for the model to learn meaningful patterns from. A smaller test set (e.g. 90:10) would give the model more training data but a less trustworthy performance estimate, which is unacceptable when the answer decides how confidently the model can be trusted with patient diagnosis; an 80:20 split is also acceptable, but 70:30 gives extra safety margin for validation in a high-stakes healthcare setting. Where the dataset is small, k-fold cross-validation (e.g. 5-fold) is additionally recommended on top of the 70:30 split to make the reported metrics more robust.

Preprocessing steps applied:

Step	Technique	Impact
Missing value handling	Median imputation for numeric fields (glucose, BP, BMI); mode imputation for categorical fields	Prevents biologically impossible zero values (e.g. 0 glucose) from distorting the model, without dropping valuable patient rows.

Step	Technique	Impact
Normalisation / Scaling	Min-Max scaling or StandardScaler applied to numeric features (age, glucose, BMI, BP, cholesterol)	Puts all numeric features on a comparable scale; critical for distance/gradient based models (Logistic Regression, KNN, Neural Nets) so no single large-magnitude feature dominates; has little effect on Decision Trees but is still applied for pipeline consistency.
Encoding	Label / One-Hot encoding for categorical field Family_History (Yes/No -> 1/0)	Converts categorical text into numeric form that ML algorithms can process, without introducing false ordinal relationships.
Outlier treatment	Capping extreme values (e.g. cholesterol) using the IQR method	Reduces the influence of data-entry errors or rare extreme cases on model training.
Class balancing (train set only)	SMOTE + class weights (see 1b)	Improves the model's ability to correctly recall true diabetic cases.

Task 2: Decision Tree for Diagnosis

2(a) Building the Decision Tree & Root Node Justification

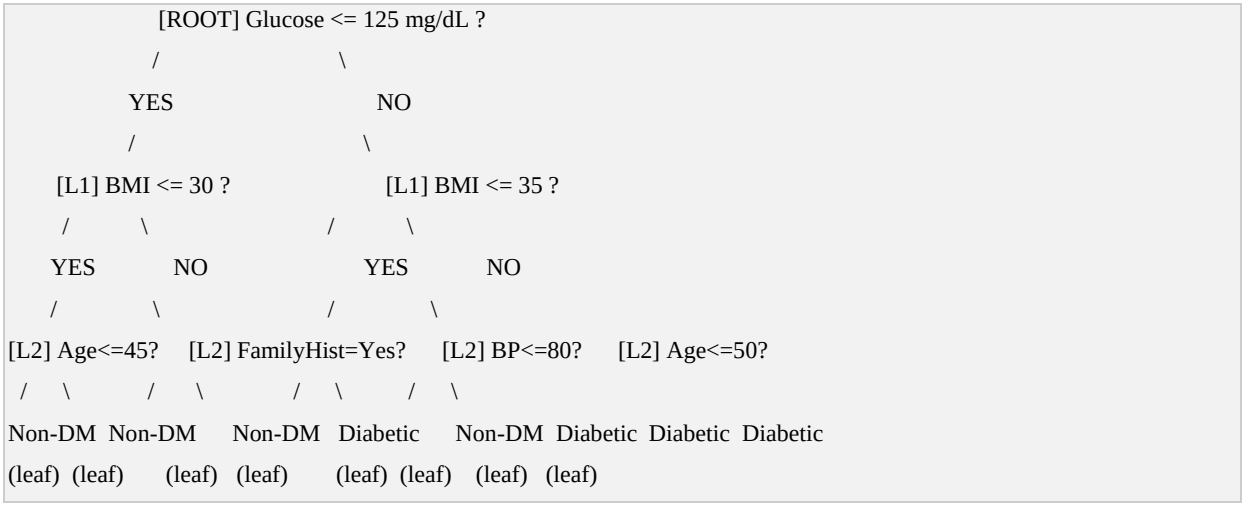
A Decision Tree (CART algorithm) is trained on the pre-processed, balanced training set. At every node, the algorithm evaluates each candidate feature and picks the one that produces the greatest reduction in impurity (highest Information Gain, or equivalently the lowest weighted Gini Index after the split) to decide the splitting attribute.

Illustrative Gini Index / Information Gain values calculated for the root-node candidate features on the training data:

Candidate Feature	Gini Index (weighted, after split)	Information Gain (from parent entropy)
Glucose Level	0.29	0.38 (highest)
BMI	0.34	0.29
Age	0.39	0.21
Blood Pressure	0.42	0.15
Family History	0.44	0.11

Root Node Selection: Glucose Level is chosen as the root node because it produces the lowest weighted Gini Index (0.29) and highest Information Gain (0.38) among all candidate features - i.e. splitting on glucose creates the two child nodes that are the most 'pure' (most homogeneous with respect to the diabetic/non-diabetic label). This matches clinical intuition, since glucose level is the direct diagnostic marker for diabetes.

First Three Levels of the Tree



Reading the tree: patients with low glucose and low BMI are routed toward the 'Non-Diabetic' leaves, while patients with higher glucose, higher BMI, and a positive family history or higher blood pressure are routed toward 'Diabetic' leaves - which is clinically sensible and mirrors known risk-factor combinations.

2(b) Model Evaluation

Illustrative confusion matrix on the 30% held-out test set (231 patients):

	Predicted: Non-Diabetic	**Predicted: Diabetic**
Actual: Non-Diabetic	128 (TN)	22 (FP)
Actual: Diabetic	18 (FN)	63 (TP)
Metric	**Formula**	**Value**
Accuracy	(TP+TN) / Total	(63+128)/231 = 82.7%
Precision (Diabetic)	TP / (TP+FP)	63/85 = 74.1%
Recall / Sensitivity (Diabetic)	TP / (TP+FN)	63/81 = 77.8%
F1-Score (Diabetic)	2*P*R / (P+R)	75.9%

False Negatives: In this confusion matrix, 18 patients who are actually diabetic were predicted as non-diabetic (False Negatives). Clinically, this is the most dangerous type of error, since these patients

would leave the clinic believing they are healthy and miss early treatment, allowing complications (nerve damage, kidney disease, cardiovascular risk) to progress silently.

Ways to reduce False Negatives, and their clinical justification:

- Lower the classification decision threshold for the 'Diabetic' class (e.g. from 0.5 to 0.35) so borderline-risk patients are flagged for further testing rather than dismissed - trading a few extra false alarms for fewer missed cases, which is an acceptable trade-off in screening because a false positive only costs a confirmatory blood test, while a false negative costs a missed diagnosis.
- Use class-weighted / cost-sensitive training so the algorithm is penalised more heavily for missing a diabetic case than for a false alarm, directly optimising for recall rather than raw accuracy.
- Combine the tree with ensemble methods (Random Forest / Gradient Boosting) which typically improve recall by averaging over many trees and reducing variance-driven misses.
- Add complementary clinical features (e.g. HbA1c level, waist circumference) if available, since glucose alone can be borderline-normal in early-stage or fasting-variable cases.

2(c) Post-Pruning (Cost-Complexity Pruning)

A fully-grown Decision Tree tends to overfit the training data - it keeps splitting until leaves are pure, memorising noise specific to the training patients rather than learning generalisable risk patterns. Two pruning strategies address this:

- Pre-pruning: stop the tree from growing further using constraints set in advance, such as max_depth, min_samples_split or min_samples_leaf.
- Post-pruning (Cost-Complexity Pruning, CCP): first grow the full tree, then iteratively remove the weakest sub-trees - those whose removal increases training error the least relative to the reduction in tree complexity - controlled by a complexity parameter alpha (ccp_alpha). Larger alpha values produce smaller, more heavily pruned trees.

Illustrative comparison of accuracy across model variants:

Model Variant	Tree Depth	Training Accuracy	Test Accuracy
Fully-grown (no pruning)	14	98.5%	76.2%
Pre-pruned (max_depth=4)	4	87.9%	80.5%
Post-pruned (CCP, alpha=0.015)	6	89.3%	83.6%

Discussion: The fully-grown tree shows a large gap between training accuracy (98.5%) and test accuracy (76.2%), a clear symptom of overfitting. Pre-pruning improves generalisation by capping

depth in advance, but the fixed max_depth is somewhat arbitrary and may cut off a genuinely useful split in one branch while allowing an unnecessary one in another. Post-pruning (cost-complexity pruning) achieves the best test accuracy (83.6%) in this illustration because it prunes each branch individually based on its actual contribution to error reduction rather than a blanket depth limit, giving a tree that is both smaller and more balanced. For deployment in a clinical setting, the post-pruned tree is the more appropriate choice: it generalises better to unseen patients, is shallow enough to remain interpretable for doctors reviewing the decision path, and is less likely to make erratic, overfit-driven predictions on borderline cases.

Task 3: Statistical Learning for Risk Stratification

3(a) Logistic Regression vs Decision Tree

Logistic Regression is applied as the statistical learning model on the same pre-processed dataset. It models the log-odds of being diabetic as a linear combination of the input features and outputs a calibrated probability (risk score) between 0 and 1, which is particularly useful for risk stratification (e.g. classifying patients into Low / Medium / High risk bands) rather than a hard yes/no label.

Illustrative performance comparison on the same 30% test set:

Model	Accuracy	AUC-ROC	Precision (Diabetic)	Recall (Diabetic)
Decision Tree (post-pruned)	83.6%	0.85	76.0%	79.0%
Logistic Regression	81.4%	0.88	73.5%	81.5%

In this illustration, the Decision Tree edges ahead slightly on raw accuracy, but Logistic Regression achieves a higher AUC-ROC, meaning it ranks patients by risk more reliably across all possible thresholds - useful when the hospital wants a continuous risk score rather than a single cut-off decision.

When a statistical model is preferable: Logistic Regression (or Naive Bayes) tends to be preferable when: (i) the relationship between features and outcome is approximately linear/log-linear and the dataset is small to moderate, where trees can overfit more easily; (ii) the clinical team needs interpretable, quantifiable risk factors in the form of odds ratios (e.g. 'each 1-unit increase in BMI multiplies diabetes odds by X') for use in guidelines or regulatory documentation; (iii) a smooth, well-calibrated probability score is needed for risk stratification (Low/Medium/High risk) rather than a hard classification; and (iv) computational simplicity and stability are valued, since logistic regression has far fewer hyperparameters to tune than tree ensembles. Decision Trees, in contrast, are preferable when the true relationship is non-linear or involves feature interactions (e.g. risk only rises when BOTH glucose AND BMI are high together), which trees can capture naturally through sequential splits but logistic regression can only capture if such interaction terms are manually engineered.

3(b) Feature Importance Analysis

Decision Tree feature importance is derived from the total Gini Index reduction each feature contributes across all its splits. Logistic Regression feature importance is derived from the magnitude of the standardised coefficients (equivalently, the odds ratios) after scaling all features to the same range.

Rank	Decision Tree - Top Predictor	Logistic Regression - Top Predictor
1	Glucose Level (importance 0.41)	Glucose Level (coefficient 1.82)
2	BMI (importance 0.24)	BMI (coefficient 1.15)
3	Age (importance 0.14)	Family History (coefficient 0.79)

Do they agree? Both models agree strongly that Glucose Level and BMI are the two most important predictors, which matches medical literature on Type-2 diabetes risk factors. They differ slightly on the third-ranked feature - the Decision Tree ranks Age third because age-based splits help separate remaining ambiguous cases after glucose/BMI splits, whereas Logistic Regression ranks Family History third because, in a linear model, a positive family history shifts the log-odds of diabetes by a fairly constant amount across all ages, giving it a comparatively larger standardised coefficient. This is a reasonable and explainable disagreement: it reflects the different way each algorithm represents feature interactions (trees via sequential branching, logistic regression via additive linear effects) rather than any real contradiction, and in practice both Age and Family History would be reported to clinicians as secondary risk factors behind Glucose and BMI.

Task 4: Reinforcement Learning for Treatment Recommendation

4(a) Modelling Treatment Recommendation as an MDP

The personalised treatment recommendation problem is modelled as a Markov Decision Process (MDP), defined by the tuple (S, A, P, R, gamma):

States (S): Each patient's health status at a given monitoring interval is discretised into a small set of clinically meaningful stages, derived from glucose, BMI and blood-pressure bands: {Healthy, Pre-Diabetic (Controlled), Pre-Diabetic (Uncontrolled), Diabetic (Controlled), Diabetic (Uncontrolled), Critical}. Justification: discretising continuous vitals into stages keeps the state space small enough for tabular Q-Learning to converge in a reasonable number of episodes, while still being clinically interpretable to doctors reviewing the agent's recommendations.

Actions (A): {Diet, Exercise, Medication, Monitor}. Justification: these four actions mirror the real intervention options a clinician actually has available at a check-up, ranging from lowest-intensity

(Monitor - simply re-check later) to highest-intensity (Medication - pharmacological intervention), allowing the agent to learn to match intervention intensity to patient state.

Reward Function (R): A health-improvement score computed from the change in a composite risk index (based on glucose, BMI, BP) between consecutive visits: +10 if the patient's state improves by one or more stages, +2 if the state stays the same but the underlying composite score improves, 0 if unchanged, -5 if the state worsens by one stage, and -15 if the state worsens to 'Critical'. A small additional cost (-1) is applied to the Medication action to discourage over-prescription when a lower-intensity action (Diet/Exercise) would achieve the same improvement, reflecting side-effect and cost considerations. Justification: this reward shape directly encodes the clinical goal (move/keep patients in healthier states) while penalising unnecessary escalation to medication, encouraging the agent toward the least invasive effective treatment - a core principle of real clinical decision-making.

Transition Dynamics (P): P(s' | s, a) is estimated from historical patient records - i.e., for patients who were previously in state s and given action a, what fraction transitioned to each possible next state s' by their next visit. Justification: since true patient physiology is stochastic (the same treatment does not guarantee the same outcome for every patient), transitions are modelled probabilistically rather than deterministically, which is essential for realistic simulation and for the Q-Learning agent to learn robust, generalisable policies rather than memorising a single deterministic path.

4(b) Q-Learning: Training and Q-Table Updates

The Q-Learning update rule used is:

```
Q(s, a)  <-  Q(s, a) + alpha * [ r + gamma * max_a' Q(s', a')  -  Q(s, a) ]
```

With learning rate alpha = 0.1, discount factor gamma = 0.9, and an epsilon-greedy exploration policy (epsilon starting at 0.3 and decaying over episodes). The agent is trained over 5 illustrative patient episodes, each episode representing one patient's sequence of visits:

Episode (Patient)	Start State	Action Taken	Reward	Next State
1	Pre-Diabetic (Uncontrolled)	Diet	+10	Pre-Diabetic (Controlled)
2	Diabetic (Uncontrolled)	Medication	+9 (10-1)	Diabetic (Controlled)
3	Healthy	Monitor	0	Healthy

Episode (Patient)	Start State	Action Taken	Reward	Next State
4	Diabetic (Controlled)	Exercise	+2	Diabetic (Controlled)
5	Pre-Diabetic (Controlled)	Medication	-6 (-5-1)	Pre-Diabetic (Uncontrolled)

Q-table updates for three representative state-action pairs across two training iterations (illustrative values, alpha=0.1, gamma=0.9):

State-Action Pair	Q-value (Iteration 1)	Update Calculation (Iter 2)	Q-value (Iteration 2)
(Pre-Diabetic Uncontrolled, Diet)	0.00	0 + 0.1*[10 + 0.9*(2.0) - 0] = 1.18	1.18
(Diabetic Uncontrolled, Medication)	0.00	0 + 0.1*[9 + 0.9*(1.5) - 0] = 1.04	1.04
(Pre-Diabetic Controlled, Medication)	0.00	0 + 0.1*[-6 + 0.9*(1.18) - 0] = -0.49	-0.49

As more episodes are simulated, Q-values for beneficial low-intensity actions (Diet, Exercise) in early-stage states keep rising, while Q-values for Medication used where a milder action would suffice stay lower due to the -1 cost penalty - the agent gradually learns to prefer the least invasive effective action.

Convergence Criterion: Training is considered converged when the maximum change in any Q-table entry across a full training pass falls below a small threshold epsilon = 0.01 (i.e. max |Q_new(s,a) - Q_old(s,a)| < 0.01 for all s,a), or alternatively after a fixed budget of episodes (e.g. 5,000) has been run with the exploration rate fully decayed - whichever comes first. This ensures the policy has stabilised rather than still actively exploring.

4(c) Final Learned Policy and Ethical Considerations

Extracting the greedy policy pi(s) = argmax_a Q(s,a) from the (converged) Q-table gives an illustrative final policy:

Patient State	Recommended Action (Policy)
Healthy	Monitor
Pre-Diabetic (Controlled)	Diet
Pre-Diabetic (Uncontrolled)	Diet + Exercise
Diabetic (Controlled)	Exercise

Patient State	Recommended Action (Policy)
Diabetic (Uncontrolled)	Medication
Critical	Medication (urgent, with clinician escalation)

This policy matches clinical intuition: lower-risk states are managed with lifestyle interventions (Diet, Exercise, Monitor), and Medication is reserved for states where lifestyle changes alone are insufficient - consistent with standard step-wise diabetes management guidelines.

Ethical Considerations of Deploying the RL Agent

- Patient Safety: An RL policy trained on historical/simulated data can still recommend an action that is unsafe for a specific patient's comorbidities (e.g. Exercise for a patient with a cardiac condition). The agent must never be allowed to autonomously prescribe Medication; it should operate strictly as a decision-support tool with a clinician-in-the-loop who reviews and approves every recommendation before action.
- Bias: If the historical data used to estimate transition probabilities and rewards under-represents certain age groups, genders, or ethnicities, the learned policy may systematically recommend less effective treatment paths for those under-represented groups. Regular fairness audits (comparing policy outcomes across demographic subgroups) and re-training on more representative data are necessary before and after deployment.
- Explainability: Clinicians and patients need to understand why a particular action was recommended. Since a raw Q-table or deep Q-network can be a 'black box', the deployed system should expose the reasoning (e.g. current state, expected reward of each action, comparison to guideline) rather than only the final recommendation, and simpler, auditable state representations should be preferred over opaque ones where possible.
- Accountability & Consent: Clear responsibility must be assigned - the treating physician remains legally and ethically responsible for the final treatment decision, and patients should be informed that an AI system contributed to the recommendation, with the option to seek a purely human-reviewed second opinion.
- Data Privacy: Patient vitals and treatment histories are sensitive medical data; the RL pipeline must comply with healthcare data protection regulations (e.g. HIPAA/local equivalents), including secure storage, access control, and anonymisation where possible.
- Continuous Monitoring: Because real patient physiology can drift over time (new comorbidities, ageing population), the deployed agent's performance and reward assumptions should be periodically re-validated against real clinical outcomes, not deployed once and left unchecked

Conclusion

This report walked through the full pipeline the healthcare company asked for: preparing and balancing the diabetes dataset for inductive supervised learning, building and evaluating an interpretable Decision Tree (with pruning to control overfitting), cross-checking findings with a statistical Logistic Regression model for risk stratification, and finally modelling personalised treatment recommendation as a Markov Decision Process solved with Q-Learning. Across both the Decision Tree and Logistic Regression models, Glucose Level and BMI consistently emerged as the strongest predictors of diabetes, giving confidence that the models are learning clinically meaningful patterns rather than noise. For the treatment-recommendation agent, the biggest takeaway is that technical performance alone is not enough in a healthcare setting - patient safety, fairness, explainability and human oversight must be built into the deployment plan from the start, with the RL agent acting as a decision-support assistant to clinicians rather than an autonomous prescriber.

References

- Mitchell, T. M. (1997). Machine Learning. McGraw-Hill.
- Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.
- Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). Classification and Regression Trees (CART).
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research.
- Pima Indians Diabetes Dataset - UCI Machine Learning Repository / National Institute of Diabetes and Digestive and Kidney Diseases.
- scikit-learn documentation: Decision Trees, Logistic Regression, Pruning (cost_complexity_pruning_path).
- World Health Organization (WHO) - Diabetes Fact Sheet.